fapolicyd for Power Users

Why · What · How — with one live demo

SELinux Development Team Workshop

October 19, 2026

How this session works

Together now

- Why it exists (vs SELinux)
- What the pieces are
- How you operate it
- Setup + one small demo

At home later

- Full lab scripts in `demos/`
- ld.so, integrity, custom rules
- SELinux dual-denial drill
- Ops / metrics deep dive

Audience

You already know Linux, packaging, audit, and SELinux. Goal: leave as fapolicyd power users — correct model first, muscle memory at home.

Agenda

- Why
- What
- How
- Together: setup + small demo
- Your turn (at home)

Two questions, two layers

SELinux — behavior
Given labels, is this access allowed?

- Domain / type / role / MLS
- How the process behaves
- Transitions & relabeling

fapolicyd — trust
Is this file a known artifact?

- RPM / manual provenance
- Path + size + SHA-256
- First-match rules

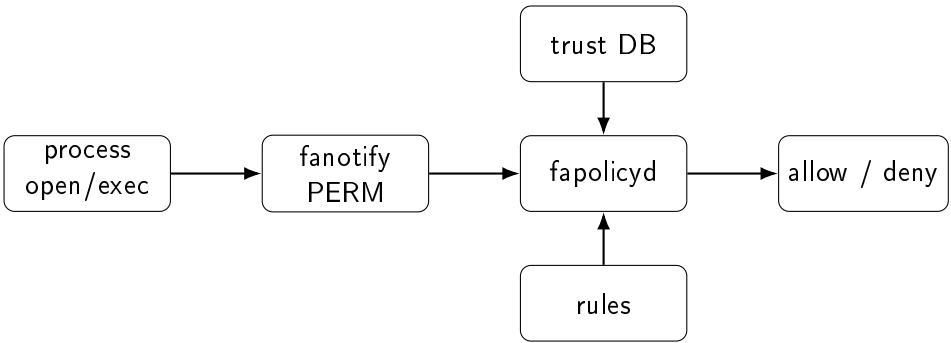
Complementary

`bin_t` on everything in `/usr/bin` is intentional for SELinux. fapolicyd is the layer that cares whether that binary is *known*.

What it is good for (honest threat model)

- Blocks untrusted code: home-dir malware, pip/npm drops, dropped shells
- Closes ld.so / interpreter tricks against unpackaged content
- Does **not** stop root from turning it off (same class as SELinux permissive / IMA off)
- Early boot reduces the race; root before login is out of scope

Architecture



Synchronous on the calling thread — latency and queue depth matter.
Never `ptrace`/`strace`/`gdb` the daemon (deadlock).

Trust database — the center of gravity

- `rpmdb` — RPM payload (filtered); DNF/RPM plugin notifies
- `filedb` — manual entries in `trust.d/`
- Entry: `type path size sha256`
- Rules match `trust=1` / `trust=0`

Power-user instinct

Prefer adding trust over writing one-off allow rules.
Cheaper lookups, clearer intent, less policy churn.

Rules — first match wins

Recipe: `decision permission subject : object`

Decisions: `allow`/`deny` + `_audit`/`_syslog`/`_log`
Permissions: `open`, `execute`, `any`

- Snippets in `rules.d/` $\xrightarrow{\texttt{fagenrules}}$ `compiled.rules`
- Default policy: `known-libs` (stay out of the way if trusted)
- Custom rules when you need *who*/*how*, not just known-ness

Canonical pattern (shared libs)

```
allow perm=open all : ftype=application/x-sharedlib trust=1
deny_audit perm=open all : ftype=application/x-sharedlib

allow perm=execute all : trust=1
deny_audit perm=execute all : all
allow perm=open all : all
```

Whitelisting by trust — not SELinux-style confinement.

Integrity (know the tradeoff)

Mode	Cost	Detects
none	—	known path only
size	free	accidental overwrite
ima	cheap	hash via IMA
sha256	high	content change

Default is none. `size` is often enough.

Full sha256 under load can hurt latency — treat as a capacity decision.

Day-to-day operations

- Inspect: `-list`, `-dump-db`, `-check-status`
- Trust: `-file add/delete/update` then always `-update`
- Validate: `-check-rules [-lint]`, `-check-trustdb`, `-check-path`
- Health: `-check-metrics`, timing start/stop, compact trust DB

Full cheat sheet + labs $\rightarrow$ take-home pack.

Safe debugging loop

1. `systemctl stop fapolicyd`
2. `fapolicyd -permissive -debug-deny`
3. Reproduce → read `rule:N dec=deny ...`
4. Map N with `fapolicyd-cli -list`
5. Fix trust (preferred) or reorder rules
6. Drop `-permissive` only when denials are intentional

Need fanotify audit events? At least one audit rule loaded (e.g. watch on `/etc/shadow`).

SELinux × fapolicyd triage

Symptom	Likely SELinux	Likely fapolicyd
AVC	type/perm	—
FANOTIFY deny	—	trust/rule
EACCES, no AVC	maybe dontaudit	check fanotify
Breaks after rpm -i	restorecon	trust not updated

Same binary

can hit **both**. Teach: `ausearch -m avc` and `ausearch -m fanotify`.

Anti-patterns

- Huge allow-lists instead of fixing trust
- Attaching a debugger to the daemon
- `integrity=sha256` without capacity planning
- `ignore_mounts` on trees interpreters can read
- Editing `compiled.rules` by hand
- Assuming SELinux allow $\Rightarrow$ fapolicyd allow

Live: setup (follow along)

Disposable VM, root, SELinux enforcing preferred.

1. `dnf install fapolicyd fapolicyd-selinux rpm-plugin-fapolicyd`
2. Ensure an audit rule exists (else `deny_audit` is quiet)
3. `systemctl enable -now fapolicyd`
4. Sanity: `fapolicyd-cli -check-status` and `-list | head`

Scripted: `sudo demos/00-lab-prep.sh`

Live: one demo — untrusted binary

The whole story in four commands:

```
cp /usr/bin/ls /opt/fapo-workshop/my-ls
chmod +x /opt/fapo-workshop/my-ls
/opt/fapo-workshop/my-ls          # DENY

fapolicyd-cli -file add /opt/fapo-workshop/my-ls \
  -trust-file workshop
fapolicyd-cli -update
/opt/fapo-workshop/my-ls          # ALLOW
```

Optional peek: `ausearch -start recent -m fanotify`
or stop service and run `fapolicyd -permissive -debug-deny`.

What we just proved

- Unpackaged copy $\neq$ trusted, even if identical bytes to an RPM file
- Trust DB is the remediation path (not a random allow rule)
- FANOTIFY denials are a different audit class than AVCs

You will configure this yourselves

After today you should be able to, without us:

1. Install, enable, and verify health
2. Trust an unpackaged `/opt` app end-to-end
3. Add a numbered `rules.d` drop-in and reload safely
4. Debug denials in permissive mode and map `rule:N`
5. Triage AVC vs FANOTIFY on the same failure

Cookbook + showcases: `TAKE-HOME.md`

Config knobs that matter

Knob	When you touch it
`trust.d/` + CLI	unpackaged software (default remediation)
`rules.d/NN-*.rules`	who/how exceptions; keep sparse
`integrity`	size / ima / sha256 tradeoff
`permissive`	soak before enforce
`syslog_format`	richer deny context (add `trust`)
caches / `q_size`	only after metrics say so
`ignore_mounts`	last resort; noexec data mounts only

Cool showcases to run yourself

1. **Clone attack** — copy of `ls` denied; trust heals it
2. **ld.so bypass** — loader cannot smuggle untrusted ELF
3. **"malware" drop** — curl/pip payload in `$HOME` dies
4. **Integrity gotcha** — mutate trusted file; size check bites
5. **Dual stack** — wrong label *and* untrusted ⇒ AVC + FANOTIFY
6. **GID gate** — trusted binary, still denied by group rule
7. **Live X-ray** — `-debug-deny` maps denials to rule numbers

Scripts: `demos/` and `demos/showcases/`

Take-home map

- `TAKE-HOME.md` — config cookbook + showcase gallery
- `demos/00–09` — structured drills
- `demos/showcases/` — flashy one-liners for demos/talks

Suggested: finish cookbook once, then pick 3 showcases to present back to the team next week.

Discussion

- staff_u / sysadm_u machines: where does fapolicyd add value?
- How do we teach customers AVC vs FANOTIFY triage?
- Image-mode / ostree: does trust storytelling change?

References

- https://github.com/linux-application-whitelisting/fapolicyd
- https://github.com/linux-application-whitelisting/fapolicyd-tests
- RHEL Security hardening — fapolicyd chapter
- Man: fapolicyd(8), fapolicyd.rules(5), fapolicyd-cli(8)

Questions

Slides + take-home:
`workshop/fapolicyd-power-users/`